# Does the Test Tell a Fix From No Fix?

Discriminating-test scoring for evaluating and training local coding agents

Gemma 4 Developer Agent Competition, Paper Track

October 2026

**Abstract**

Repository-level coding benchmarks score an agent's patch by running tests after it and checking that they pass. The rule never asks whether those tests failed before the patch. We measure how often that matters on the 129 public tasks of the Gemma 4 Developer Agent Competition by running every task's target tests before and after the reference fix under the competition's own pytest settings. Of the 669 required tests the grader extracts from the test patches, 199 (30%) already pass on the unfixed code, 22 tasks name no required test at all, and 2 tasks are resolved by an empty patch. Across all tests in the target files, only 1,490 of 7,153 (21%) distinguish a fix from no fix, and 62 tasks hinge on a single such test. We propose scoring on these *discriminating* tests in two forms: a strict pass/fail that an empty patch cannot earn, and a continuous credit, the share of discriminating tests that pass, which gives a dense signal for comparing configurations and for training. On two submissions with known answers, the gold patch scores 112 of 125 under the competition rule and 110 strictly, the empty patch 2 and 0. We release per-task annotations, a scorer, and two difficulty strata, entry distance and tool answerability, so that evaluations can report where a model fails, not only how often.

## 1 Introduction

The competition marks a task resolved when, after the agent's patch, pytest on the files touched by the task's test patch exits 0 and every required test passes [1]. Required tests are the task's `FAIL_TO_PASS` list when it exists and otherwise the test functions the test patch adds. On the public set `FAIL_TO_PASS` is empty for 124 of 129 tasks, so extraction is the rule in practice.

That rule is a reasonable acceptance test for a human-written fix. As an *evaluation metric* it has two gaps. It credits tests that pass whatever the patch does, so a model can score without changing behaviour. And it is a single bit per task, so it says nothing about a patch that fixes two of three failing behaviours, exactly the signal a researcher comparing configurations, or a trainer shaping a reward, needs.

Both gaps are measurable. We run each task's target tests on the base commit with the test patch and again after the reference fix, and classify every test by its outcomes. A test that fails or cannot be collected before and passes after is **discriminating**. We contribute the first measurement of test discriminativeness for this benchmark (§4); a scoring rule with three outputs, validated on the gold and empty patches (§3, §5); a protocol for using it to test and improve models (§6); and the annotations and scorer.

## 2 Related work

**Pass/fail hides failure modes.** On $\tau^2$-bench, 4-bit Gemma-4-31B doubled its out-of-list tool calls with no change in final score, because the error budget absorbed the damage [2]. Repeat-run consistency metrics such as pass^k exist for similar reasons [3].

**Weak tests inflate benchmarks.** SWE-bench+ found solution leakage and tests that do not check the fix [4]; models localize SWE-bench Verified files from the issue text alone far better than on fresher sets [5]. These works audit benchmark contents; we audit the grading rule and offer a replacement.

**Training wants denser signal.** The latest survey of issue resolution lists fine-grained process rewards and intrinsic validation through tests among open problems [6]. Loss on high-value segments of 10% of trajectories beat training on all of them [7], and rewarding the discovery of gold locations lifted small localizers above larger ones [8]. All of these need per-task signal finer than one bit.

# 3 Definitions

For a task, run pytest on the target files on the base commit plus test patch (*pre*) and after the reference fix (*post*). Record each test's outcome in each run, *uncollected* when it does not appear.

- **Effective requirement**: every test collected post-fix in the target files (exit 0 needs all of them), plus the extracted names, flagged *named*.
- **Discriminating test**: pre is failed, error or uncollected, and post is passed.
- **Null-patch resolvable**: the pre run already meets the competition rule.

| Score | Rule | Property |
|---|---|---|
| Competition rule | exit 0, at least one pass, every named test passed | reproduces the grader |
| Strict | competition rule, and all discriminating tests pass (at least one exists) | an empty patch scores 0 |
| Credit | share of discriminating tests that pass | continuous in $[0, 1]$ |

**Table 1:** The three scores computed for a submission on a task.

# 4 What the public set looks like

Runs use per-task virtual environments under Python 3.13.5 with the competition's pytest flags; 114 of 129 tasks run cleanly in both phases and 125 have both runs.

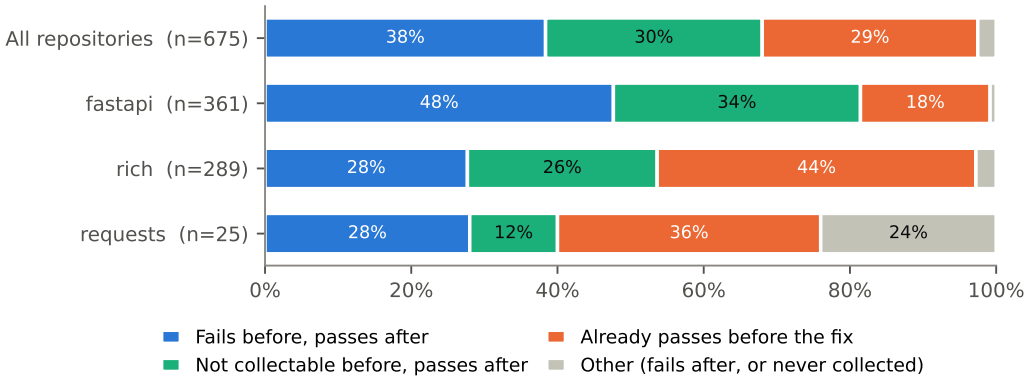



**Figure 1:** Outcome before and after the reference fix of every test the grader requires (named tests), pooled and per repository.

| Repository | Named | Fail, then pass | Uncollectable, then pass | Pass before fix | Other |
|---|---|---|---|---|---|
| fastapi | 361 | 172 (48%) | 122 (34%) | 64 (18%) | 3 |
| rich | 289 | 80 (28%) | 75 (26%) | 126 (44%) | 8 |
| requests | 25 | 7 | 3 | 9 | 6 |
| All | 675 | 259 (38%) | 200 (30%) | 199 (29%) | 17 |

**Table 2:** Named required tests by outcome. "Other": fails after the fix, or never collected (6).

**Named required tests.** Of 675 named entries, 259 fail before the fix and pass after, 200 cannot be collected before (their module imports code the fix adds) and pass after, 199 already pass before the fix, and 17 fail after or never appear (Figure 1, Table 2). In rich, 44% of the tests the grader requires pass on the unfixed code.

**Per task.** 22 tasks name no test (fastapi 14, rich 6, requests and httpx 1 each), so exit 0 is their whole requirement. Of the 103 tasks that name tests, the discriminating share is 1.0 for 58, strictly between 0 and 1 for 42, and 0 for 3; the mean is 0.76 (95% CI 0.70 to 0.82). Counting every test in the target files, 1,490 of 7,153 discriminate. Figure 2 shows the distribution: 62 tasks hinge on one discriminating test and 7 have none.

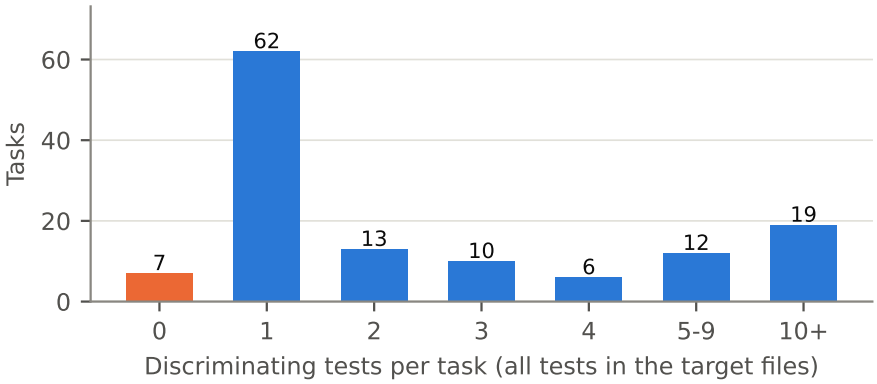



**Figure 2:** Discriminating tests per task, counting every test in the target files (125 tasks with both runs).

Four of those 7 are requests tasks whose tests did not collect in our environment; three are real (rich_4079, fastapi_14851, fastapi_14077).

**Null-patch resolvable.** Two tasks. In fastapi_14851 the three added tests exercise behaviour the base commit already has, so all pass before the fix and pytest exits 0. In fastapi_14077 the test patch only rewrites assertions from a status-code constant to the literal 422, no test name is extracted, and the target file is green before the fix.

# 5 Validating the scorer

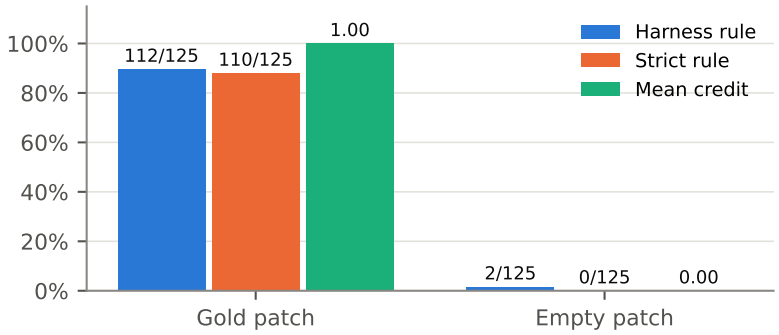



**Figure 3:** The two reference submissions under the three scores (125 tasks with both runs).

We score two submissions whose correct answers are known: the gold patch (the post run) and the empty patch (the pre run). The gold patch resolves 112 of 125 tasks (89.6%) under the competition rule and 110 (88.0%) strictly, with mean credit 1.00; the empty patch resolves 2 (1.6%) under the competition rule, 0 strictly, with credit 0.00 (Figure 3). The strict rule removes exactly the two free tasks from the gold patch's score and nothing else. The 13 tasks the gold patch fails under the competition rule are environment failures in our runs (collection or import errors, or no collectable tests); the official cached dependencies would close most of them.

# 6 Using the metric

**To test a model,** report four numbers: the competition rate, the strict rate, mean credit and the empty-patch floor. Stratify by two annotations from our companion audit of the provided code graph: *entry distance*, the minimum call-graph hops from where the failing test enters the library to the function the fix changes, and *tool answerability*, whether a perfect user of the provided graph tools could reach every gold function.

Figure 4 shows the strata: 49 of 91 usable tasks are answerable, and none of the 22 with no executed or reachable gold node are. A tool intervention can then be credited where it is able to help. Credit adds power to paired comparisons: a condition that fixes two of three discriminating tests scores 0.67 instead of 0, and 60 of 125 tasks have two or more discriminating tests.

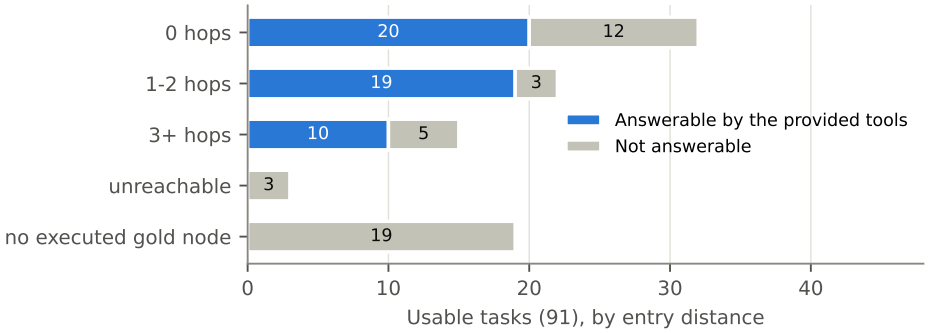



**Figure 4:** The 91 usable tasks by entry distance, split by whether the provided graph tools could in principle surface every gold function.

**To improve a model:**

1. filter supervised fine-tuning trajectories to those reaching credit 1 on tasks with two or more discriminating tests, removing free points and single-test coin flips;
2. use credit as a reinforcement-learning reward, which cannot be earned by leaving the code unchanged;
3. order a curriculum by entry distance, and train tool-adoption interventions on the answerable stratum first.

**For benchmark builders,** accept a task only if it has at least one discriminating test, preferably two, and is not null-patch resolvable. The hidden set is built by the same pipeline; the check costs two pytest runs per task.

# 7 Limitations

The metric needs a pre-fix run, which only the benchmark owner can do on hidden tasks: it is a benchmark-side annotation, not something an agent computes. Our runs used network-installed dependencies, so discriminativeness is undefined, not zero, for tasks that did not collect. We count uncollectable-before tests as discriminating because the grader would see a non-zero exit; a stricter reading drops 200 named tests to "unknown". With two free tasks in 129, the strict rule moves today's public score by at most 1.6 points; the case rests on the unguarded rule, the training signal and the unchecked hidden set. The uses in §6 are proposals; we have not yet scored agent runs.

# Reproducibility

Annotations and reference scores are in `planA_results/vm-a2/results/a9/` (`task_metrics.csv`, `task_tests.jsonl`, `a9_summary.json`, `scores_gold_vs_null.json`), produced by `scripts/planA/a9_metrics.py` from the traces in `vm-a2/results/a2_runs/`; `papers/make_figures.py` draws every figure.

# References

[1] Competition organizers. swegemma evaluation harness reference, Section 8 (Phase 2 verification), 2026.

[2] Jang et al. Flat score, amplified failures: How the error budget masks damage in quantized LLM agents, 2026. https://www.alphaxiv.org/abs/2607.27275

[3] Xu et al. The devil is in the interface: How tool architecture shapes coding agent behavior. COLM, 2026. https://www.alphaxiv.org/abs/2608.11386

[4] Aleithan et al. SWE-Bench+: Enhanced coding benchmark for LLMs, 2024. https://consensus.app/papers/details/a2fde21b76f5544c9edca661a2b6e868/

[5] Prathifkumar et al. Does SWE-Bench-Verified test agent ability or model memory?, 2025. https://consensus.app/papers/details/b4db7508813d5b5bb87f40a948b9ddbf/

[6] Li et al. Advances and frontiers of LLM-based issue resolution in software engineering: A comprehensive survey, 2026. https://www.alphaxiv.org/abs/2601.11655

[7] Zheng et al. SWE-Prime: Fewer trajectories, better performance, 2026. https://www.alphaxiv.org/abs/2608.27449

[8] Kim et al. $A^2$Agent: Action-aware RL for code localization agents, 2026. https://www.alphaxiv.org/abs/2608.29831